Penetration Testing Report: Scanning and Enumeration

Name: [Humphrey Wanjala]
Date: [21/07/2026]
Role: Junior Penetration Tester

Reconnaissance Summary

- What was the objective of your reconnaissance process?

  To independently perform network reconnaissance and service enumeration against a Metasploitable 2 target simulating a healthcare provider's staging environment, identifying exposed services and misconfigurations ahead of a scheduled external audit.

- What network range did you scan?

  10.0.2.0/24 — the VirtualBox NAT Network segment shared between the Kali attacker VM and the Metasploitable2 target.

- What scanning strategy did you choose (e.g., ping scan vs. full scan), and why?

  A phased approach: first a ping/host discovery scan to confirm the target was live and locate its current IP, followed by two full-port scans (-p-) — one aggressive (-A) for maximum detail, and one stealth (-sS -T2) for comparison. This reflects real-world practice of confirming scope before choosing a scan profile matched to the client's monitoring sensitivity.

- Were there any challenges identifying the target system?

  Yes. Initial scans were mistakenly run from the target machine (Metasploitable2) against itself rather than from Kali, producing false "host seems down" results. Separately, the target's IP address changed between VM reboots due to DHCP reassignment, requiring a fresh discovery scan from Kali to relocate it before scanning could proceed correctly.

Nmap Scan Results (Host Discovery + Port Scanning)

- What Nmap scan(s) did you run? *(Include flags used and why you chose them)*

sudo nmap -A -p- 10.0.2.7 -oN aggressive_scan.txt

sudo nmap -sS -T2 -p- 10.0.2.7 -oN stealth_scan.txt

- How many hosts responded?

  1 host responded to full scanning (the Metasploitable2 target). A prior subnet-wide discovery scan (sudo nmap -sn 10.0.2.0/24) found 4 active hosts total on the segment: the NAT gateway, DHCP server, Kali itself, and the target.

- What were the IP address and hostname of the target?

  IP: 10.0.2.7 — Hostname: metasploitable

- What open ports and services were discovered?

Port	Service	Version
21	FTP	vsftpd 2.3.4
22	SSH	OpenSSH 4.7p1
23	Telnet	Linux telnetd
25	SMTP	Postfix smtpd
53	DNS	ISC BIND 9.4.2
80	HTTP	Apache 2.2.8
111	RPCbind	
139/445	SMB	Samba 3.0.20
512/513/514	rlogin/rexec	
1099	Java RMI	
1524	Ingreslock	
2049	NFS	
3306	Mysql	5.0.51a
5432	PostgreSQL	8.3

5900	VNC	
6000	X11	
6667	IRC	UnrealIRCd
8009/8180	Tomcat/AJP	

- 🔎 *Screenshot or Nmap output snippet required*

Service Enumeration and Risk Indicators

- Choose at least **two services** from your scan to enumerate further.
    - For each:
        - What tool(s) did you use for enumeration?

          enum4linux -a 10.0.2.7, smbclient -L //10.0.2.7 -N, rpcclient -U "" 10.0.2.7

        - What additional information did you gather (e.g., share names, OS details, banners)?

          Confirmed Samba version 3.0.20; identified that anonymous (null session) authentication was accepted, allowing share names and system details to be enumerated without valid credentials.

        - Were any risky misconfigurations discovered? (e.g., anonymous FTP, outdated version, Samba misconfig)

          Yes — anonymous SMB access was permitted, exposing share listings and configuration details to unauthenticated users

        - How might an attacker exploit these misconfigurations in a real scenario?

          An attacker could use the anonymous session to map internal share structure and gather reconnaissance, then exploit the underlying Samba 3.0.20 usermap_script vulnerability to achieve remote command execution as root, without needing valid credentials at any stage.

- Identify potential CVEs or vulnerabilities tied to version numbers or known exposures.

  CVE-2011-2523 — vsftpd 2.3.4 backdoor, allows unauthenticated remote root shell.

  CVE-2007-2447 — Samba 3.0.20 usermap_script command injection, allows unauthenticated remote code execution as root.

  Unrelated to CVE tracking but equally severe: port 1524 (Ingreslock) runs a pre-existing root shell listener requiring no exploitation at all.

Attack Surface Analysis

- Based on your scan and enumeration:
  - What does the attack surface of this system look like?

    Extremely broad and severe — nearly every listening service corresponds to a known, publicly documented, high-severity vulnerability, consistent with a deliberately unpatched legacy Linux system used for training purposes.

  - Which services are the most vulnerable and why?

    vsftpd 2.3.4 and Samba 3.0.20 are the most critical — both offer unauthenticated, direct paths to root-level remote code execution with no chaining or privilege escalation required. Port 1524 (Ingreslock) is equally severe since it exposes an already-listening root shell.

  - What services would you target in a follow-up exploitation phase?

    vsftpd 2.3.4 first, followed by Samba 3.0.20.

  - Justify your choice using findings from your enumeration or known vulnerabilities.

    vsftpd 2.3.4's backdoor (CVE-2011-2523) requires no authentication bypass or memory corruption — simply connecting with a specially crafted username yields an immediate root shell, making it the fastest, lowest-effort path to full compromise. Samba's usermap_script flaw (CVE-2007-2447) is the logical secondary target given its similarly direct, unauthenticated RCE path, confirmed reachable during SMB enumeration.

Network Scan Strategy Reflection

- Did you change your scan type at any point (e.g., added `-sS`, increased verbosity)?

  Yes — after initial scans run from the wrong machine (the target itself) returned false negatives, I corrected course by running all subsequent scans from Kali. I also re-ran host discovery after the target's DHCP-assigned IP changed between VM sessions.

- How did different scan techniques affect your results?

  The aggressive scan (-A) returned significantly richer data — service versions, OS fingerprinting, and NSE script output — in one pass. The stealth scan (-sS -T2) returned a much narrower dataset (primarily open/closed port states) due to its slower timing and lack of version/script probing, but generated far less detectable traffic.

- What trade-offs exist between stealth and aggressive scans?

  Aggressive scanning trades stealth for speed and depth of information — ideal for time-constrained assessments on networks without active monitoring, but likely to trigger IDS/IPS alerts. Stealth scanning trades completeness and speed for reduced detection risk — better suited to live production environments or engagements where avoiding tip-off to defenders is a priority, at the cost of needing follow-up scans to gather version/script data.

Lessons Learned

- What were your biggest takeaways about scanning and enumeration?

  Scan source and target verification matter as much as scan syntax — a correct nmap command still fails if run from the wrong machine or against a stale IP. Confirming reachability (ping) before scanning saves significant troubleshooting time. I also learned that Metasploitable2's vulnerabilities compound quickly — multiple independent, unauthenticated root-level exploits exist simultaneously across different services.

- What would you do differently in a real-world pentest environment?

I would confirm client-approved scan windows and scope before any scanning begins, start with stealth-profile scans by default, and escalate to aggressive scanning only once authorized or once initial recon confirmed minimal active monitoring. I would also flag and report severe findings like the vsftpd backdoor immediately to the client rather than holding them for the final report, given the immediate root-compromise risk they represent.